ResolveAI

Technical Architecture and Design

An incident investigation copilot: it gathers evidence from read-only systems, produces cited root-cause hypotheses and drafts an RCA for a person to verify. Nothing is executed automatically.

Version	1.0, 31 October 2026
Describes	Branch **feature/dotnet-port**, commit **a247662**
Stack	.NET 10, Blazor Server, ASP.NET Core, SQLite, MCP, Claude
Status	Working demo with synthetic data. Live Claude not yet run for real.

Contents

1. Executive summary and reader's guide 3

Part A: As-built, verified against the code

2. Architecture 5
3. Workflows 7
4. Components 10
5. MCP servers 12
6. Data model and API 14
7. Design decisions 16
8. Security and threat model 17
9. Deployment and operations 19
10. Testing 21

Gaps and roadmap

11. Risks, limitations and gaps 22
12. Roadmap 23

Part B: Future target architecture, a plan that is not built

13. Target architecture 24

Appendices

A. Configuration, glossary and samples 28

How to read the diagrams



Each page header shows which kind of content it holds: AS-BUILT, GAPS AND ROADMAP or PLANNED: NOT BUILT. Statements in Part A were checked against the source. Nothing in Part B exists yet; products named there are options to integrate and none of them have been tested.

1. Executive summary

The problem

During an incident, on-call engineers jump between logs, deployment history, dashboards, runbooks and old postmortems while the clock runs. AI assistants can help, but one that invents log lines, cites nothing or changes production is worse than none.

The approach

ResolveAI pulls the evidence for one incident into a single workflow and asks Claude to reason only over that evidence. Three rules shape the design:

- **Evidence first.** Code gathers the evidence and picks similar past incidents. Every record gets a typed ID, and the server removes any citation it can't match to a real record.
- **Read-only by construction.** Each system sits behind its own MCP server with fixed, parameterised checks. No tool runs arbitrary SQL or shell commands.
- **A person decides.** Output is labelled as a draft, checks are recommendations, and an incident stays in a review queue until someone records a verified cause and reviews the RCA.

Current status

Working today

- Blazor Server UI and HTTP API on one ASP.NET Core app
- Full investigation pipeline in recorded mode
- Four read-only MCP servers, with 17 tools, including real SQL Server checks
- RCA drafting, editing, versioning and Markdown export
- 205 automated tests, plus a Docker image

Not done or not proven

- **Live Claude has never run against the real API.** It is only tested against a fake API.
- Data is synthetic, apart from optional SQL Server checks
- No user accounts; a shared access code gates paid runs
- CI workflow has not yet run on GitHub
- Single process, with SQLite

Key risks

Risk	Why it matters	Section
Quality of live analysis is unproven	The core value claim rests on Claude choosing useful checks and reasoning well over real evidence. Recorded mode can't show this.	11, 12
Evidence leaves the network in live mode	Redaction is pattern-based and can miss a secret. On-premises sites need an explicit decision about this.	8, 13.4
No sign-in or per-user permissions	The app is safe only because it binds to localhost. It can't be shared as it stands.	8, 13.2

Where the design is heading

Part B describes a plan for an on-premises deployment: one Windows Server, SQL Server, Windows sign-in with Active Directory roles, Claude reached by direct API or private cloud link per site, and more evidence sources added as further read-only MCP servers.

Reader's guide

If you are	Read	You will be able to
A new engineer	Sections 3, 4, 5, 6, 10	Understand, run and safely change the system
An architecture reviewer	Sections 2, 7, 11, 13	Judge the design, its trade-offs and its risks
A hiring or demo audience	Sections 1, 2	See what was built and the engineering thinking behind it
An enterprise adopter	Sections 8, 9, 11, 12, 13	Assess security, operations and what production would take

2. Architecture

2.1 System context

ResolveAI sits between an on-call engineer and the systems they would otherwise check by hand. It only reads from those systems, and only calls out to Claude in live mode.

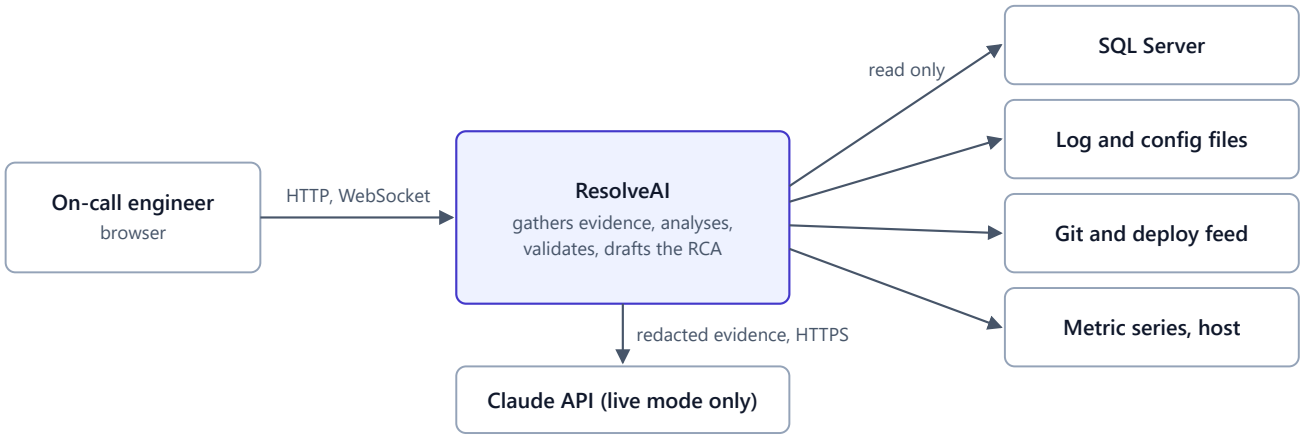


Figure 1. System context. All connections to the systems on the right are read-only.

2.2 Containers and processes

One ASP.NET Core process serves both the UI and the API. It starts one child process per system when it boots and talks to each over MCP on stdio.

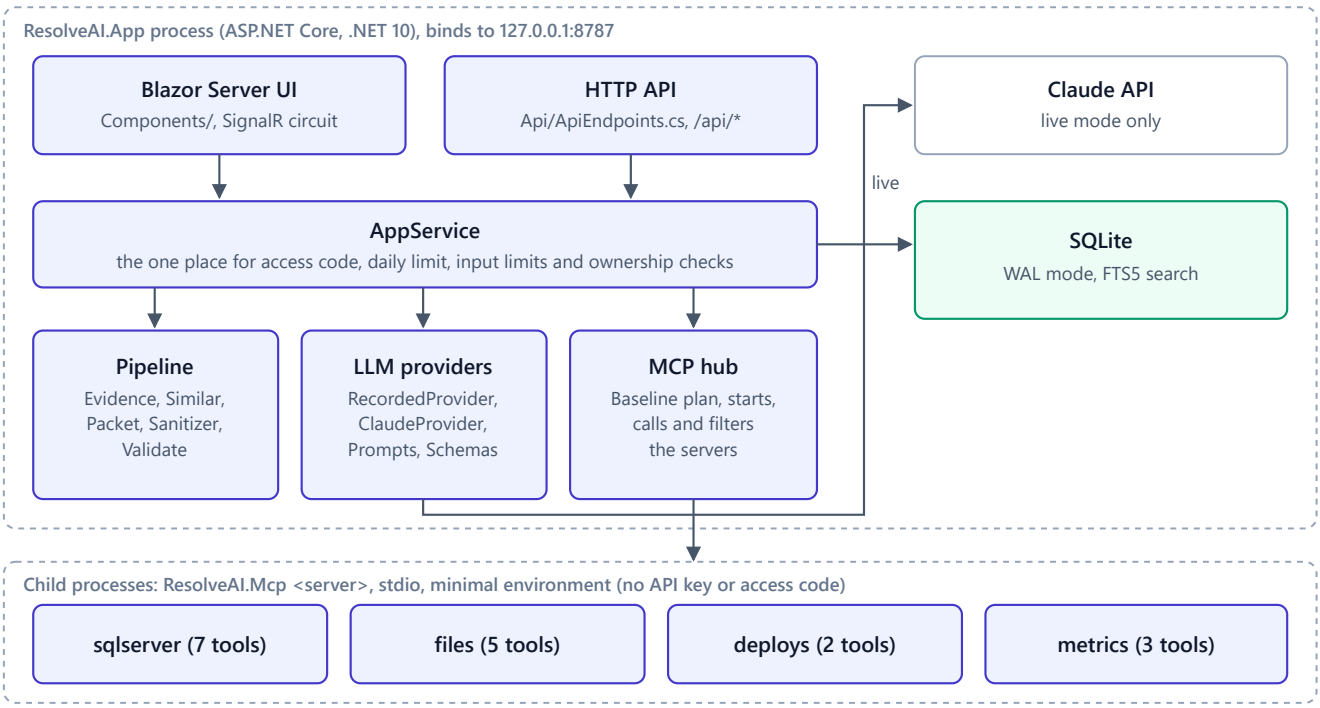


Figure 2. Containers. The UI calls AppService directly; the HTTP API exists for scripts, the export route and compatibility.

2.3 Technology

Concern	Choice	Why
Runtime	.NET 10, ASP.NET Core	One language across UI, API and tools
UI	Blazor Server, interactive, no prerender	Secrets and logic stay on the server; no separate front-end build
Styling	Tailwind 4.3.3; generated CSS committed	Running the app needs only the .NET SDK
Data	SQLite through Microsoft.Data.Sqlite and Dapper	No database to install; FTS5 gives keyword search
LLM	Official `Anthropic` .NET SDK, model `claude-opus-5-5`	Structured output, tool runner, server-side fallback
Tool protocol	`ModelContextProtocol` SDK over stdio, pinned to version 2025-11-25	A standard boundary; servers also work from Claude Code
SQL Server access	Microsoft.Data.SqlClient	Windows sign-in or a SQL login, with one driver
Tests	xUnit, WebApplicationFactory, Microsoft.Playwright	API and real-browser tests in the same language

2.4 Project layout

```
ResolveAI.sln
  src/ResolveAI.Contracts/   DTOs, JSON rules, evidence IDs, JavaScript-compatible formatting
  src/ResolveAI.Mcp/         executable: ResolveAI.Mcp <sqlserver|files|deploys|metrics>
  src/ResolveAI.App/
    Api/                     HTTP routes and error mapping
    Components/              Blazor pages (Pages/), incident views (Incident/), shared parts (Shared/)
    Data/                    SQLite access, migrations, demo seed
    Llm/                     providers, prompts, output schemas
    Mcp/                     hub and baseline check plan
    Pipeline/                evidence, similar incidents, packet, redaction, validation
    Services/                AppService, investigations, RCA, incidents, access code store
  tests/ResolveAI.Tests/     unit, API, MCP, golden and attack tests
  tests/ResolveAI.E2E/       browser tests on a temporary database and a free port
  demo-systems/              synthetic logs, deploy feed, metrics, SQL Server setup and scenario
  tools/css/                 regenerates the committed CSS (the only use of Node)
```

3. Workflows

3.1 Investigation, end to end

Starting a run returns HTTP 202 at once and the work continues on a background task. Each stage's timing and counts are stored as a trace, shown in the UI as *How this was produced*. The UI polls every 1.5 seconds.

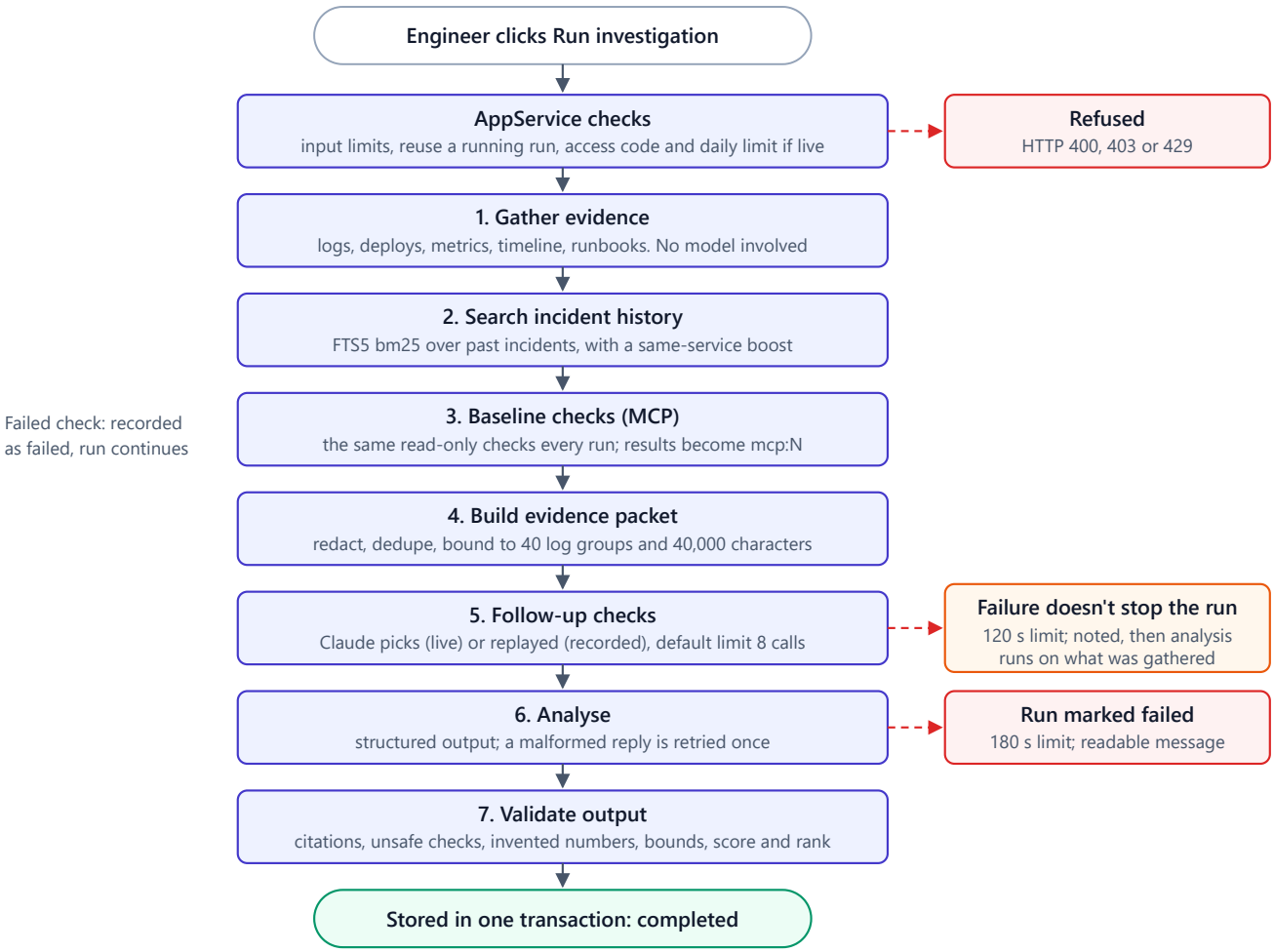


Figure 3. Investigation workflow. Stages 1 to 4 and 7 involve no model; only stages 5 and 6 depend on the provider.

3.2 Live follow-up loop

This is what lets Claude ask further questions of the systems. The SDK tool runner drives the loop, but every call passes through ResolveAI's callback, which records it and enforces the limit. The model can't exceed it, because past the limit it only receives a refusal.

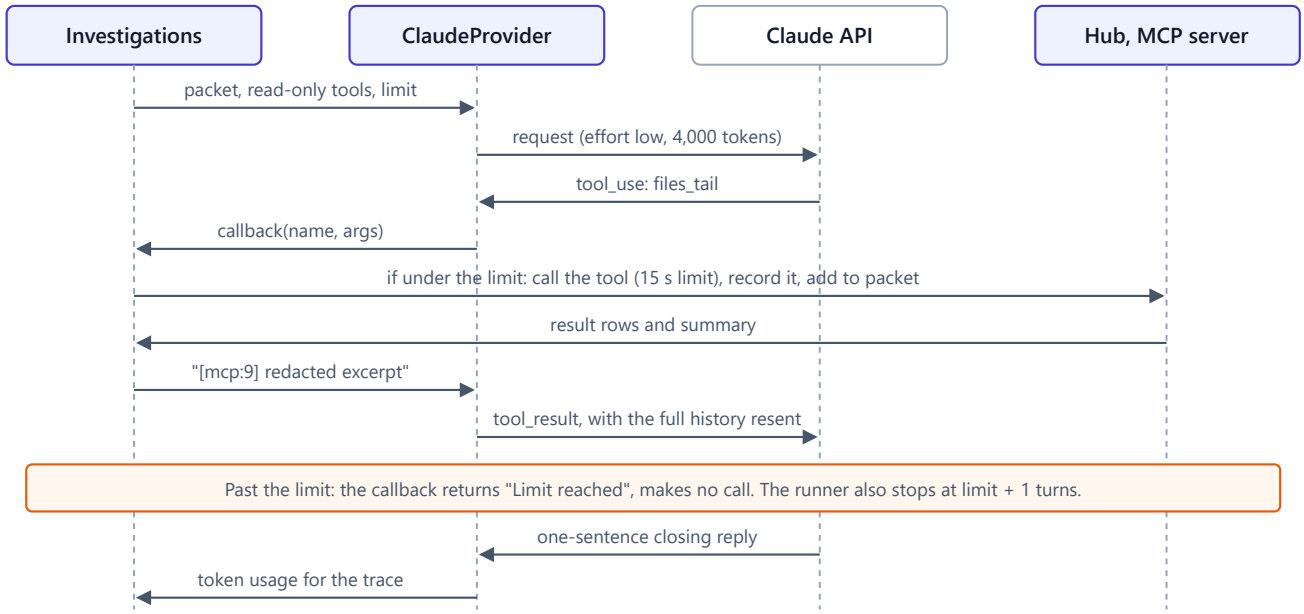


Figure 4. Follow-up sequence. Tested against a fake API. It has not yet run against the real Claude API.

3.3 Engineer review, through to the RCA

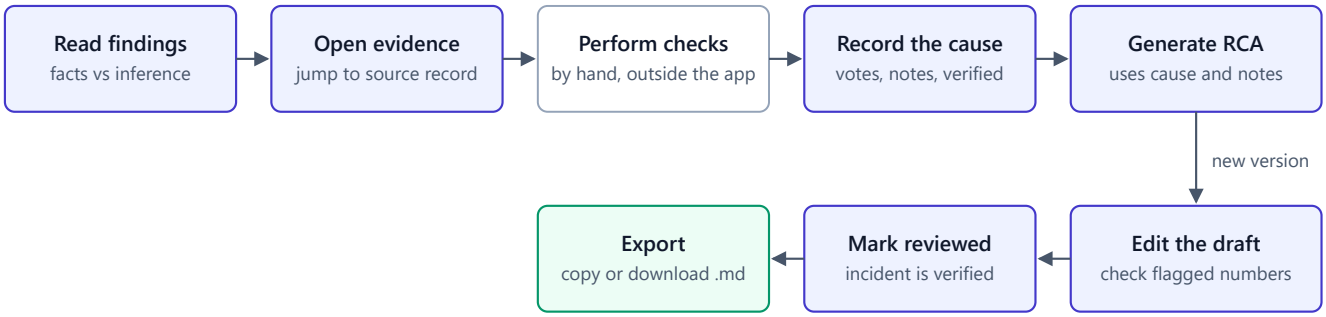


Figure 5. Review workflow. Generating again creates a new version and never overwrites a person's edits.

3.4 State lifecycles

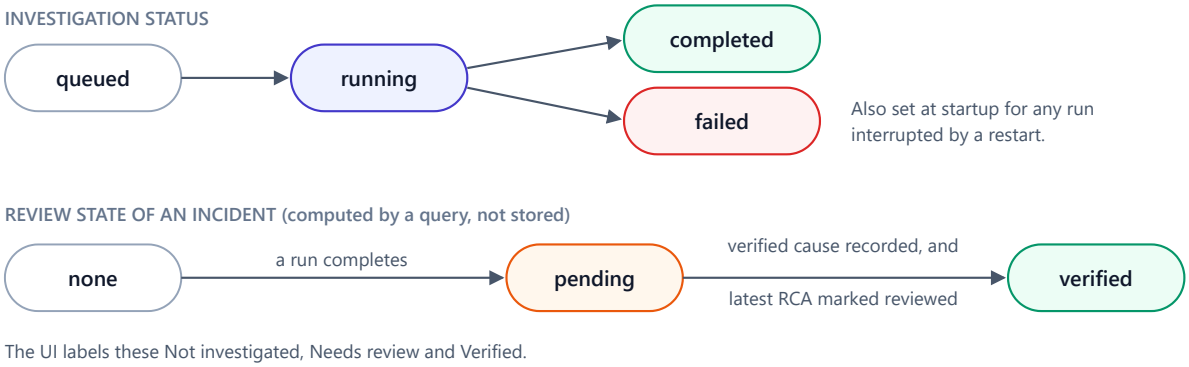


Figure 6. State lifecycles.

3.5 Time limits

Operation	Limit	On expiry
Analysis stage, and RCA draft	180 s each	Run fails, or RCA request errors, with a readable message
Follow-up stage	120 s	Noted under validation; analysis continues
One Claude HTTP request	90 s, 2 retries	Mapped to a readable error
MCP server startup	20 s	System shows *Not working*; the app still starts
One tool call at the hub / inside the server	15 s / 10 s	Failed result recorded; the run continues
Git command; SQL connect and command	8 s; 5 s and 8 s	Tool returns an error result; git is killed

4. Components

4.1 Evidence gathering

`Pipeline/EvidenceLoader.cs` is deterministic and involves no model. For one incident it loads:

- **Logs:** the affected service, or linked to the incident, from 30 minutes before the start to one hour after detection, up to 200 rows.
- **Deployments:** the same service in the 48 hours before, and other services within one hour either side.
- **Also:** metric snapshots, timeline events, and up to five runbooks or guides, plus error signals counted from the logs by signature.

4.2 Similar incidents

`Pipeline/Similar.cs` builds search terms from the title, description, error signatures and same-service change summaries, then runs an FTS5 query. The model never chooses candidates; it can only explain them.

```
score = -bm25 x 10 + matched terms + (3 if same service)
keep   candidates with at least 2 matched terms, and at least 35% of the best score, up to 5
```

4.3 Evidence packet

`Pipeline/Packet.cs` builds the only thing the model is allowed to see. Each item has a typed ID that the validator later checks citations against.

Prefix	Source	Prefix	Source
`log:`	Log group	`doc:`	Runbook or guide
`dep:`	Deployment	`hist:`	Past incident
`met:`	Metric snapshot	`tl:`	Timeline event
`mcp:N`	System check result		

- **Deduplication:** log lines differing only by IDs or numbers collapse into one group with a count.
- **Bounds:** 40 log groups, preferring errors and warnings; excerpts of 600 characters, or 1,200 for tool results; 40,000 characters in total, dropping information logs first.
- **Ordering:** by time, with tool results last in call order.

4.4 Redaction

`Pipeline/Sanitizer.cs` runs when the packet is built. Stored logs and tool rows aren't changed, so an engineer still sees the raw record, and a database trigger rejects any update to log rows. It redacts URL credentials, `Bearer` and `Basic` values, provider-style keys (`sk_` , `sk-ant-` , `AKIA` , `ghp_`), `key=value` secrets, email addresses and card-like numbers.

> **Limitation:** these are patterns. A secret in an unusual format can pass through, which matters most once real data is sent to a cloud model.

4.5 Validation

`Pipeline/Validate.cs` enforces what a JSON schema can't express, because model output is treated as untrusted.

Check	Behaviour
Citations	Any ID not in the packet is removed and noted. A hypothesis left with none is marked **speculative** and forced to low confidence.
Unsafe checks	Recommended checks describing destructive actions (`rm -rf` , `kubectl delete` , `DROP TABLE` , `terraform destroy` and similar) are removed.
Invented numbers	Numbers with a unit, or of 10 or more, that don't appear in the evidence are flagged, not deleted, so a person decides.
Similar incidents	Explanations for anything outside the retrieved candidates are ignored.
Bounds	5 hypotheses, 8 list items, 5 checks, and clipped text lengths.
Ranking	`2 x confidence (1, 2 or 3) + min(citations, 5) - counter-evidence` . For ordering only; never shown as a probability.

4.6 LLM providers

RecordedProvider (default)

No network. INC-1042, INC-1045 and INC-1046 replay written analyses, which adapt to the checks that actually ran. Other incidents get a rule-based analysis built from the packet. Output goes through the same validation as live output.

ClaudeProvider (live)

Analysis: effort medium, 16,000 tokens, structured output, `fallbacks: "default"` . Follow-ups: effort low, 4,000 tokens, through the tool runner. Replies are also checked for shape and valid confidence values before use. Refusals, cut-offs and API errors become readable messages.

4.7 MCP hub

`Mcp/Hub.cs` starts each enabled server, keeps only tools marked `readOnlyHint` , routes calls, and restarts a crashed server once. A system that fails to start shows as *Not working* and never stops the app. `Mcp/Baseline.cs` decides the fixed checks for each incident from `mcp.config.json` : database checks if the service has a database, error and warning searches of its log folders, recent deployments and its key metrics.

5. MCP servers

All four servers live in one executable, `src/ResolveAI.Mcp`, selected by an argument. Each uses `ToolServer`, which applies declared-input checks, a timeout, the read-only annotations and consistent error results to every tool.

5.1 Tools

System	Tool	What it does
Database SQL Server	`db_health`	Sessions and blocked requests per database, top waits, memory and CPU
	`db_blocking`	Blocking chains, head blocker and its last statement
	`db_long_queries`	Requests running at least *n* seconds
	`db_deadlocks`	Deadlocks from the system_health session
	`db_errors`	Errors from system_health and the optional resolveai_errors session
	`db_space`	File sizes, state and log reuse for an allow-listed database
	`db_failed_jobs`	Failed SQL Agent job steps
Files & logs	`files_search`	Plain-text search, optionally within a time window
	`files_tail`	Last lines of a text file
	`files_recent_changes`	Files modified since a time
	`files_disk_space`	Free and used space on the drive
	`files_check`	Exists, size, modified time, readable, writable
Deploys & Git	`deploys_recent`	Feed deployments and git commits in a window
	`deploys_change_summary`	Files changed and a short patch
Metrics & host	`metrics_list`	Metric names for a service
	`metrics_series`	Baseline, peak and timing in a window
	`metrics_host`	Current CPU, memory and uptime

5.2 Limits on every tool

10-second timeout, 24-hour time window, 200 rows, 64,000 characters of output (trailing rows are dropped to fit, and marked truncated), and 500 characters per value.

5.3 Read-only guarantees

System	Guarantee	How
All	No arbitrary commands or SQL	Fixed tools only. The hub also ignores any tool not marked read-only.
SQL	The server itself refuses writes, even for an admin connection	Each statement runs inside `EXECUTE AS LOGIN = 'resolveai_reader'` and always reverts. That login has `VIEW SERVER STATE`, `db_datareader` on the demo database, and read access to job history.
	Nothing a caller sends becomes SQL	Fixed text and typed parameters. The one exception is a database name, matched against an allow-list and bracket-quoted.
Files	Only inside allowed folders	The real path of every segment is resolved, so `..`, symlinks, junctions, sibling folders with the same prefix, device paths and alternate data streams are refused.
	Can't be used to read huge or binary files	Walks skip links and stop at 500 files. Files over 20 MB and binary files are skipped; tail reads at most 1 MB.
Git	No injection	No shell, fixed arguments, repositories only from config, commit must be 4 to 40 hex characters, 8-second kill, 4 MB output cap.
Metrics	A name can't become a path	Service names must be plain alphanumeric names with hyphens.

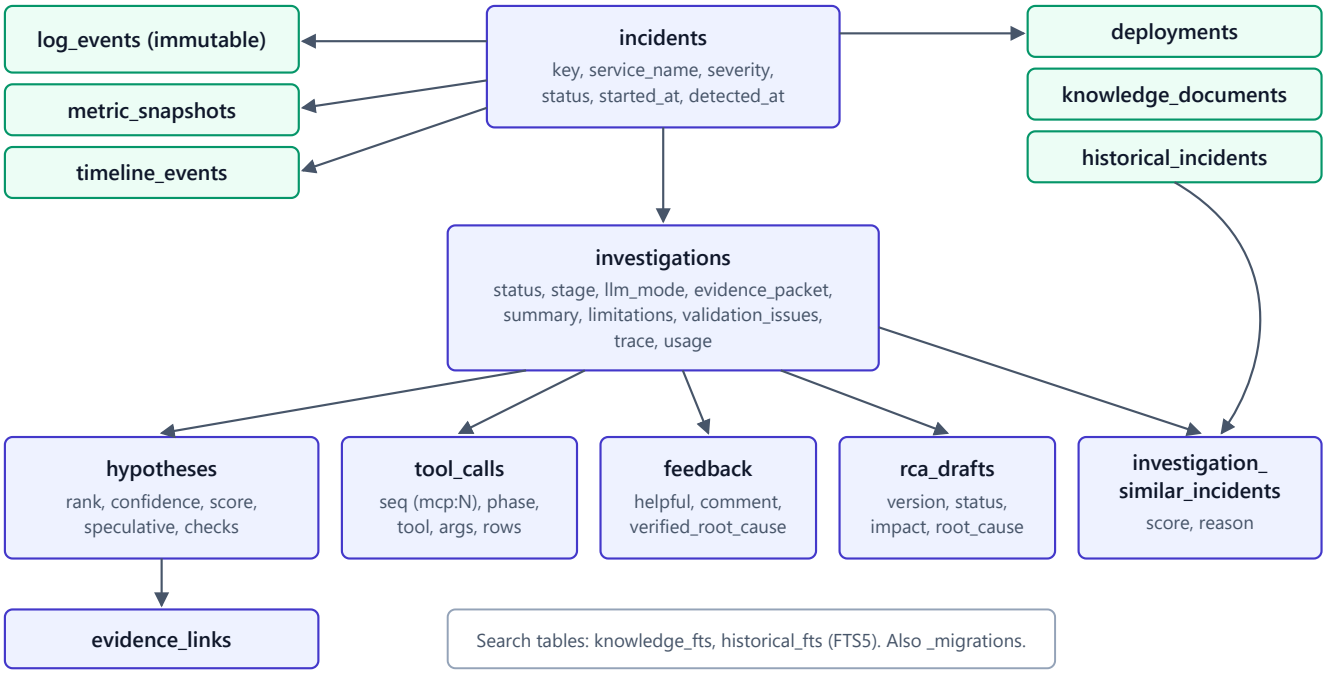
5.4 Adding a system

1. Add `<Name>Server.cs` in `src/ResolveAI.Mcp` that returns a `ToolServer`.
2. Register it in `ServerDefs` in `Program.cs`.
3. Add its colour to `SystemStyle` in `Components/Shared/Ui.cs`.
4. Add its checks to `Mcp/Baseline.cs`.
5. Add attack tests for anything a caller controls.

6. Data model and API

6.1 Data model

There are 14 tables, two FTS5 search tables and a `_migrations` table. IDs are UUID strings, and seeded IDs are derived from stable names, so recorded analyses can cite them and resets produce identical data.



Green: evidence and history. Purple: results. Also: knowledge_documents to historical_incidents, incidents to rca_drafts, hypotheses to feedback.

Figure 7. Main tables and relationships. Arrows point from a parent to the rows that depend on it.

JSON columns: lists and structured values are stored as JSON text, such as `evidence_packet`, `trace`, `limitations`, `recommended_checks` and `rows`. The stored packet is the permanent record of exactly what the model saw.

Migrations: SQL files embedded in the app and applied in filename order, each in a transaction, and recorded by filename.

6.2 HTTP API

JSON uses snake_case. Errors are `{ "error": "message" }`. `:id` on incidents accepts a UUID or a key such as `INC-1042`.

Route	Purpose and notable statuses
`GET /api/config`	Mode, live availability, usage today, follow-up limit
`GET /api/systems`	Status, detail and tools for each MCP system
`GET /api/incidents`	List and filter: `status, severity, service, q, review, page, page_size`
`GET /api/incidents/:id`	Detail, latest run, all runs, RCA versions. 404 if unknown
`GET /api/incidents/:id/evidence`	Logs, signals, deployments, metrics, timeline, runbooks

Route	Purpose and notable statuses
`POST /api/incidents/:id/investigations`	`{question?, mode?}`. 202 started; 400 bad input or live not set up; 403 wrong code; 429 over the daily limit
`GET /api/investigations/:id`	Status, stage, hypotheses with evidence, trace, tool calls
`POST /api/investigations/:id/feedback`	`{hypothesis_id?, helpful?, comment?, verified_root_cause?}`. 201
`POST /api/incidents/:id/rca-drafts`	`{investigation_id}`. 201; 400 wrong incident; 409 not completed; 502 model error
`GET`, `PATCH /api/rca-drafts/:id`	Read or save edits; only listed fields can change
`GET /api/rca-drafts/:id/export.md`	Markdown download
`GET /api/knowledge/:id`	Runbook or historical RCA document
`POST /api/demo/reset`	Reseed. 403 unless localhost or access code; 404 if demo mode is off; 409 while a run is active

6.3 UI pages

Route	Content
`/`	Incident queue with filters and the Needs review toggle
`/incidents/{id}`	Header, run controls and tabs: signals and timeline, logs, deployments, AI findings, system checks, similar incidents, RCA
`/systems`	Status and tools of each system, and how investigations use them
`/settings`	Mode, access code, demo reset

7. Design decisions

Each record gives the decision, the reasoning, what was rejected, and what it costs.

D1. Code gathers evidence; the model only reasons over it

Why: a model that searches freely can miss evidence or be steered by injected text, and its results are hard to audit. **Rejected:** a fully agentic investigation. **Cost:** baseline checks are fixed, so unusual incidents depend on follow-ups.

D2. One read-only MCP server per system, with fixed tools

Why: a separate process per system gives a clear boundary, independent failure and reuse from other MCP clients, and safety that doesn't depend on model behaviour. **Rejected:** a generic "run SQL" or "run command" tool, which would be more flexible but unsafe. **Cost:** each new question needs a new tool.

D3. Enforce read-only in the target system too

Why: running queries as `resolveai_reader` means SQL Server refuses writes even if the code has a bug or connects as an admin. **Cost:** setup needs an administrator once, and the login needs specific grants.

D4. Validate model output on the server

Why: structured output guarantees shape, not truth. **Rejected:** trusting citations, or deleting flagged text outright; flagged numbers are kept so a person decides. **Cost:** permissive matching means a number appearing anywhere in the evidence counts as verified.

D5. Similar incidents are retrieved by code

Why: deterministic and testable, and the model can't invent a precedent. **Rejected:** vector search, which adds infrastructure not justified at this scale. **Cost:** keyword search misses matches phrased differently.

D6. Recorded mode as the default

Why: demos and tests behave the same every time, with no cost or network. **Cost:** it can look more finished than it is: recorded analyses were written by hand and prove the pipeline, not the model's judgement.

D7. Blazor Server, with the UI and API sharing AppService

Why: one language and no client-side secrets, and rules that sat in route handlers moved to one place so the UI can't bypass them. **Rejected:** a Blazor UI calling the API over HTTP, which adds a hop in the same process. **Cost:** each user holds a live server connection, so scaling out would need sticky sessions.

D8. Port from Node against recorded golden files

Why: the highest risk was silent change, such as weakened redaction or different scoring. Recording every route, investigation, prompt and tool result first made differences into test failures. **Cost:** intentional behaviour changes now require updating the golden files.

D9. Pin the MCP protocol version

Why: an unpinned client probes for a newer version on a short timeout, then falls back to a handshake that a slow-starting server refuses. That made systems randomly fail on a busy machine, in about 40% of parallel test runs. ResolveAI ships both ends, so nothing needs negotiating. **Cost:** newer protocol features need a deliberate change.

D10. SQLite

Why: anyone can run the project from the README with no database setup, and FTS5 is enough at this scale. **Cost:** limited to one server; Part B replaces it.

D11. Store evidence as JSON on the investigation

Why: an investigation stays reproducible and auditable even after source data changes. **Cost:** large rows, and queries can't easily reach inside them.

8. Security and threat model

8.1 Assets and trust boundaries

The main assets are incident evidence (which can contain secrets and personal data), credentials (API key, SQL login, access code), the integrity of monitored systems, and the paid API budget.

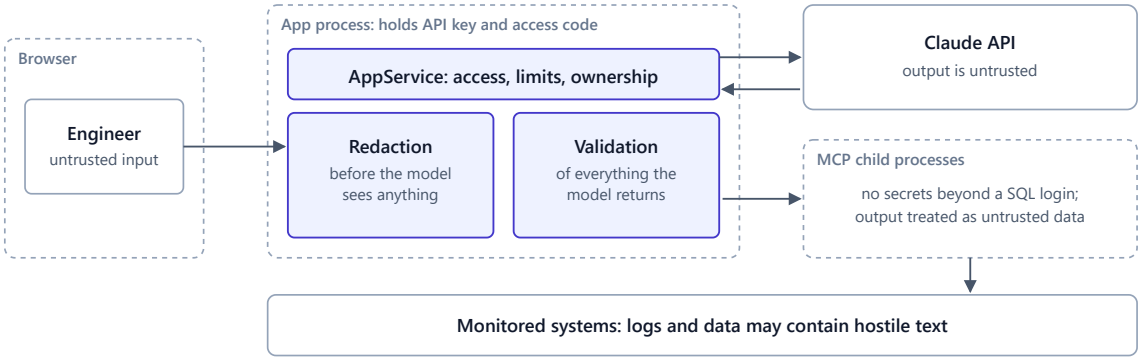


Figure 8. Trust boundaries. Everything arriving from the browser, the model or a monitored system is treated as untrusted.

8.2 Threats and mitigations

Threat	Mitigation	Tested by
Prompt injection in logs or tool results	The prompt marks evidence as data. Tools are read-only, so a successful injection can't change a system. Citations and unsafe checks are validated; a person reviews everything.	API tests
Secrets sent to the model	Redaction when the packet is built, including tool output. Pattern-based, so not complete.	Unit, golden
Destructive advice or invented facts	Unsafe recommended checks are stripped, and nothing is executed. Unknown citations are removed; unverified numbers are flagged.	Unit, API
Reading files outside the allowed folders	Real-path confinement in every segment.	Attack tests
Argument or SQL injection	No shell, hex-only commits, fixed SQL, allow-listed database names, a validated login name.	Attack tests
Writing to monitored systems	No write tools, and the reader login enforces it in SQL Server.	Real SQL test
API key reaching child processes	Environment inheritance is off, with an explicit allow-list.	Real child process
Runaway cost	Access code, daily limit (default 20 model calls), call limit per run. The limit is enforced by ResolveAI, not the model.	API tests
HTML or script in logs	Blazor encodes all text; raw markup rendering isn't used.	Browser test
Another site connecting to the Blazor circuit	Origin check on `/_blazor`.	Browser test
Access code exposure	Compared in constant time on hashes, and encrypted in the browser's storage.	Browser test

8.3 Known gaps

- **No user accounts.** Safety relies on binding to localhost. Anyone reaching the port can read everything and run recorded investigations.
- **No cross-site protection on the HTTP API.** A web page you visit could send a request to it on localhost, including `POST /api/demo/reset`, which erases investigations. Set `DEMO_MODE=false` where that matters.
- **No DNS-rebinding protection.** The Origin check compares against the Host header, and allowed hosts aren't restricted.
- **Pattern-based redaction** can miss secrets. Raw rows are shown to anyone using the app, by design.
- **No audit log of who did what,** because there are no identities.

9. Deployment and operations

9.1 Local

```
dotnet run --project src/ResolveAI.App      # http://localhost:8787
```

Requires the .NET 10 SDK. An empty database is seeded on first start. Settings come from `.env` if present, and real environment variables win over the file.

9.2 Container

```
cp .env.example .env              # optional, for live runs
docker compose up -d --build
docker compose ps                 # wait for "healthy"
```

- **Image:** published app on `dotnet/aspnet:10.0`, non-root, no Node. The app is its own health check (`--healthcheck`), since the image has no curl.
- **Network:** published on `127.0.0.1` only. Compose sets `DEMO_RESET_ANY_IP=true`, because callers arrive from the bridge network; this is safe only with that loopback binding.
- **Data:** SQLite on the named volume `resolveai-data`.
- **SQL Server from Docker:** use the `tedious` driver setting with a SQL login, as Windows sign-in isn't available.

9.3 Configuring systems

`mcp.config.json`, or the committed example if it's absent, maps each service to log folders, a git repository, a database and key metrics, and sets allowed folders. Relative paths resolve against the file's folder. Secrets stay in `.env`.

```
sqlcmd -S localhost -E -C -i demo-systems/sqlserver/setup.sql
# then: "sqlserver": { "enabled": true, "driver": "msnodesqlv8", "execute_as": "resolveai_reader" }
```

Both driver names now use `Microsoft.Data.SqlClient`; they are kept so that existing config files stay valid.

9.4 Operations

Task	How
Check health	`GET /api/config`; the Systems page shows each system's status
Back up	Copy `data/` while stopped, or `docker compose cp resolveai:/app/data ./backup`
Reset demo data	Settings page. This erases investigations, feedback and RCA drafts
Upgrade	`docker compose up -d --build`. Migrations apply at startup
After a restart	Runs in progress are marked failed; run them again

9.5 Troubleshooting

Symptom	Likely cause
A system shows *Not working*	Read its detail on the Systems page. For SQL Server, the service may be stopped or `setup.sql` not run.
Blank page in a published build	Content root or client script missing. Publish with source present, so restore sees the `.razor` files.
Live option not offered	Needs both `ANTHROPIC_API_KEY` and `LIVE_ACCESS_CODE`.
Access code forgotten after a rebuild	Expected: the keys protecting it live inside the container.
Port 8787 already in use	Another instance is running. Set `API_PORT` or stop it.

10. Testing

The suite has 205 test cases: 196 in `tests/ResolveAI.Tests` and 9 in `tests/ResolveAI.E2E`. The former also pass on Linux, inside the Docker build.

```
dotnet test tests/ResolveAI.Tests               # no external services needed
MSSQL_TEST=1 dotnet test tests/ResolveAI.Tests  # also runs against local SQL Server
dotnet test tests/ResolveAI.E2E                 # real browser, temporary database, free port
docker build --target test .                    # the same tests on Linux
```

10.1 What each layer proves

Layer	Covers	Doesn't prove
Golden files	25 route responses, 7 investigation runs with packets and RCAs, 37 tool results, all tool schemas, the seed data, and the exact prompts and output schemas match the original implementation	That the original behaviour was right
API behaviour	Citations, retries, failures, access code, daily limit, review states, input limits	Real model quality
Attack tests	Path escapes through `..`, junctions, deeper segments, sibling folders, device paths and streams; non-hex commits; unsafe login names	Unknown attack forms
Real processes	Servers start as children; the API key and access code don't reach them	
Real SQL Server (opt-in)	Every `db_*` tool runs as the reader; writes are refused; login reverts	Behaviour under production load
Fake Claude API	Request contents, schema, fallback, tool loop, iteration limit, error mapping, using the real SDK	**Whether the real API accepts and answers these requests well**
Browser	The full journey, filters, HTML in logs staying inert, encrypted access code, phone layout, refused cross-site connections	Other browsers (Chromium only)

10.2 Golden files

`tests/ResolveAI.Tests/Golden` holds what the Node implementation returned. Comparisons normalise only what legitimately varies: random IDs, current times, file modified times and sizes, and durations. To change behaviour on purpose, update the golden file in the same change, and explain why in its review.

10.3 Bugs this strategy found

Each of these passed in-process tests and only failed under more realistic conditions, which is why several layers exist:

- All MCP servers crashed at launch from the app's folder (found by running the published app).
- Blank page in the container, because restore ran before the source was copied (found by testing the image).
- Random startup failures from MCP version negotiation (found by running projects in parallel).
- The database code was never committed, because `.gitignore` matched it on Windows (found by testing a fresh clone).

11. Risks, limitations and gaps

11.1 Not proven

Item	Status
Live Claude	Never run against the real API, in the Node or .NET versions. No API key is configured. The port's request format is tested, but not real answers, cost, latency or the quality of the checks Claude picks.
Recorded analyses	Written by hand. They prove the pipeline and UI, not the value of the analysis.
GitHub CI	The workflow hasn't run on GitHub. Its Docker checks and the unit tests were run locally on Linux; the C# browser tests ran only on Windows.
Accuracy	No evaluation set exists to measure how often the top hypothesis is right.

11.2 Limitations

Area	Limitation	Addressed in
Data	Synthetic incidents, logs, deploys and metrics. Only SQL Server checks can be real today, and file and git sources once pointed at real paths.	Phase 5, 6
Evidence timing	Time-windowed checks look at the incident window, but `db_blocking`, `db_health` and `db_long_queries` show the state right now, and a summary can link the two. That can mislead for an incident that has already ended.	Phase 1
Identity	No accounts, roles or audit trail.	Phase 2
Storage	SQLite: one server, and JSON columns are hard to query.	Phase 3
Availability	One process; a restart fails runs in progress.	Accepted
Retrieval	Keyword search only.	Not planned
Confidence	Qualitative and uncalibrated.	Phase 1
Documentation	`docs/ResolveAI-Simple-Guide.pdf` still gives the old `npm` commands.	Phase 4

12. Roadmap

The phases are ordered so that the riskiest assumption is tested first: if live analysis isn't useful, the rest isn't worth building. There are no dates or effort estimates, because those depend on team size. Each phase has an exit condition that can be checked.

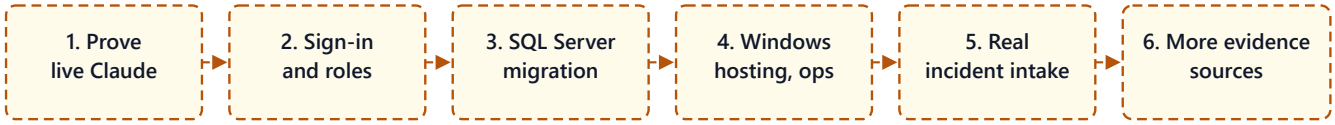


Figure 9. Roadmap phases, in order.

Phase	Delivers	Exit criteria
1. Prove live Claude	Real runs on the demo incidents and the blocking scenario. An evaluation set of incidents with known causes. Clear labelling of point-in-time evidence. Cost and latency recorded in the trace.	The evaluation runs repeatably, and its results are judged good enough to continue. Measured cost per investigation is known. Refusals and errors are seen handled in real use.
2. Sign-in and roles	Windows authentication, AD groups mapped to four roles, attribution on feedback and reviews, an audit log. Removes the shared access code. Fixes the cross-site gap.	Each role is tested for what it can and can't do, on the UI and API. Cross-site requests are refused. Every change is attributed to a person.
3. SQL Server migration	Schema and data access on SQL Server, full-text search replacing FTS5, retuned similarity scoring, a data migration tool.	Golden tests pass on SQL Server, with any scoring differences reviewed and accepted. A copy of existing data migrates and verifies.
4. Windows hosting and operations	Install on Windows Server, service accounts, secrets handling, monitoring, backup and restore, runbook. Updated guide.	A clean install follows the written steps. A restore is rehearsed and meets its targets. Alerts fire on a forced failure.
5. Real incident intake	One incident tool connected, read-only, replacing the demo seed. Model access routes for the site, with an audit log of data sent.	Real incidents appear with no manual entry. A pilot team has used it on real incidents and reports on its usefulness. Security has approved what leaves the network.
6. More evidence sources	Further read-only MCP servers, one at a time, in the order the pilot shows matters most.	For each source: attack tests pass, the read-only guarantee is documented, and it's shown to improve investigations.

> **Stopping point:** phase 1 is a real decision. If the evaluation shows that live analysis doesn't help enough, later phases should change, rather than building infrastructure around an unproven core.

13. Target architecture

> **This part is a plan. None of it is built or tested.** It records the direction chosen for an on-premises deployment, so that reviewers and adopters can assess it. Products are named as integration options, and their details should be confirmed at design time for each phase.

13.1 Overview and hosting

ResolveAI runs on one Windows Server inside the company network, using the existing Windows, Active Directory and SQL Server skills of an on-premises team.

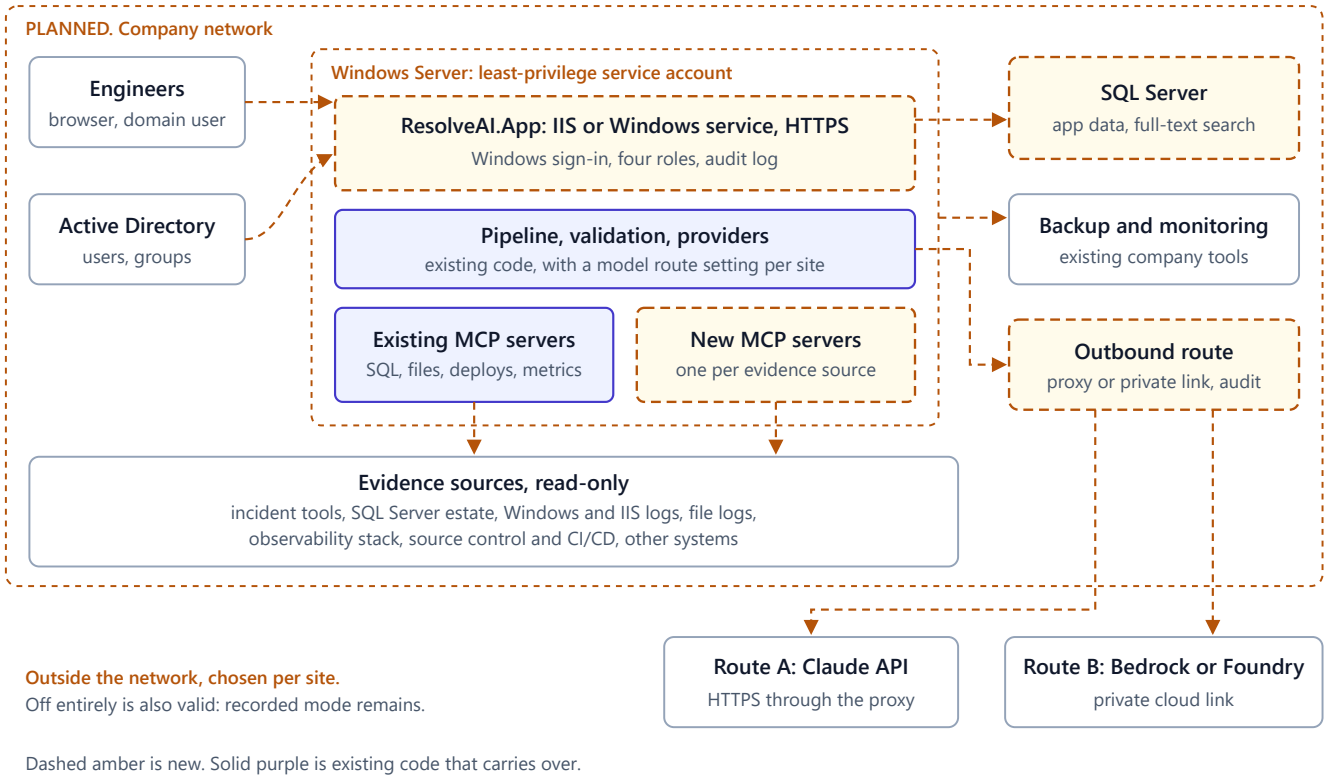


Figure 10. Target architecture on one Windows Server. Planned.

Topic	Design
Host	ASP.NET Core under IIS or as a Windows service, on HTTPS with a company certificate. IIS needs its WebSocket feature enabled for Blazor.
Service account	A dedicated low-privilege account, ideally a group managed service account. MCP servers run as children under it, with the same minimal environment as today.
Secrets	Stored outside source and config, for example in environment settings protected by Windows DPAPI or a company vault, and readable only by the service account.
Data protection keys	Persisted to a protected folder, so sessions survive restarts.

13.2 Identity and roles

Windows authentication signs users in automatically, and AD groups map to four roles. This replaces the shared access code: permission to spend on live runs becomes a role, and `Caller` in `AppService` carries a real identity instead of a code and a localhost flag.

Permission	Viewer	Investigator	Reviewer	Admin
Read incidents, evidence and findings	Yes	Yes	Yes	Yes
Run recorded investigation, add notes and votes		Yes	Yes	Yes
Run live investigation (within the daily limit)		Yes	Yes	Yes
Generate and edit RCA drafts		Yes	Yes	Yes
Record a verified root cause, mark an RCA reviewed			Yes	Yes
Change systems, model routes and limits; read audit log				Yes

- **Enforcement:** in `AppService`, so the UI and API can't differ. The API also authenticates by Windows sign-in, which closes today's cross-site gap together with anti-forgery checks.
- **Audit:** a new append-only table recording who did what and when, including what evidence was sent to a model.

13.3 Data

SQL Server replaces SQLite, on an existing instance or a dedicated one, and connects using the service account.

Area	Change	Risk to manage
Schema	Port the 14 tables. Text IDs and ISO time strings can move to proper types.	Code compares times as strings today, so change them together.
Search	SQL Server full-text search replaces FTS5.	Its rank isn't bm25, so the score formula and 35% cut-off must be retuned against known cases.
Data access	`Db.cs` and its SQL, mostly standard. `LIMIT` and similar syntax need porting.	Golden tests catch differences.
Migration	A one-way tool, run once, with row counts verified.	Keep the SQLite file until verified.
Retention	A policy for investigations and packets, which may hold sensitive evidence.	Needs agreement with the company's data owner.
Separation	App data in its own database, separate from any monitored database.	The read-only login must not reach app data.

13.4 Model access

Claude can't run on-premises, so live analysis sends evidence outside the network. Each site chooses a route in configuration, behind the existing `ILlmProvider` layer. The rest of the pipeline doesn't change.

Route	How it works	Suits
A. Direct API	HTTPS to the Claude API through the company proxy, with one allow-listed destination.	Sites allowing controlled outbound access; simplest
B. Private link	Claude through Amazon Bedrock or Microsoft Foundry over a private connection, under the company's cloud contract.	Sites requiring no public internet path
Off	No model calls. Rule-based analysis only, with all evidence gathering still working.	Sites not yet approved

Controls on every route: redaction before anything leaves; an audit record of each packet sent; live mode allowed per service, so sensitive ones can stay off; the daily limit; and automatic fallback to recorded mode with a clear label.

> **Design points for route B:** the C# SDK has dedicated clients for these platforms, and model IDs can differ. The `fallbacks: "default"` setting used today isn't available on Bedrock or Foundry, so refusal handling needs the SDK's client-side fallback instead. Availability of each feature must be confirmed per platform and tested, since structured output and the tool runner are both relied upon.

> **Decision each site must make:** redaction is pattern-based, so some sensitive content can leave the network. This should be an explicit, recorded approval by the site's security owner, not a default.

13.5 Evidence sources

Every source follows the same pattern as today: its own read-only MCP server, fixed tools, the same limits, and its results become cited evidence. File logs already work through the existing files server, by adding folders to its allow-list.

Category	Options	Purpose and notes
Incident tools	ServiceNow, Jira Service Management, PagerDuty	Import real incidents to replace the demo seed. Read-only: ResolveAI never updates tickets.
Windows and SQL estate	Windows Event Logs, IIS logs, Performance Monitor, more SQL Server instances	Closest fit for an on-premises team. Each SQL instance needs its own reader login.
Observability	Splunk, Elastic, Prometheus, Grafana	Sites that centralise logs and metrics. Queries come from fixed templates, not free text.
Source control and CI/CD	Azure DevOps Server, GitHub Enterprise, Jenkins	Real deployment history and code changes, replacing the demo feed.
File logs	Existing files server	Available now. Configure allowed folders and a service account with read-only access.
Other systems	Message queues, Kubernetes, feature flags, HTTP probes	Added through the checklist below.

Checklist for adding any system

1. Use a read-only credential, so the source system enforces the guarantee itself.
2. Provide fixed, parameterised tools, never a free-form query or command.
3. Apply the standard limits: time window, rows, output size and timeout.
4. Treat its output as untrusted, and confirm that redaction covers its formats.
5. Add attack tests for every input a caller controls, and a test against a fake source.
6. Add its baseline checks, and show that it improves investigations before keeping it.

13.6 Operations

Area	Plan
Monitoring	Health endpoint, MCP server status, run failures, model errors, spend against the limit, fed to the company's monitoring through OpenTelemetry or Windows event logs.
Logging	Structured application logs, without evidence content or secrets.
Backup	Standard SQL Server backups, plus configuration and protected keys. Recovery targets are set by the site, and restores are rehearsed.
Patching	A scheduled window, with a maintenance notice. Migrations apply at startup, after a backup.
Capacity	Each user holds a live connection and each run uses background work. To be measured in the pilot; the design expects tens of users, not thousands.

What choosing a single server accepts

- Downtime during patching or a server failure, until it is restored.
- Investigations in progress are lost on restart, and are marked failed to be run again.
- Users reconnect after a restart.

This is reasonable because ResolveAI supports incident response but isn't part of the production path: engineers can still investigate by hand while it's down. If it became critical, the next step would be a second server, which needs sticky sessions, shared keys, and investigations moved to a database-backed queue.

13.7 Open questions

These can only be settled with a real deployment:

- What accuracy from the evaluation in phase 1 is good enough to continue?
- Which incident tool does the first pilot use, and how are its services mapped to log folders, databases and metrics?
- Which route does the first site approve, and what must be redacted beyond today's patterns?
- How long may investigations and evidence packets be retained?
- IIS or Windows service, given the site's standards?
- What budget should the daily live limit reflect, once phase 1 has measured the cost?

Appendix A. Configuration, glossary and samples

A.1 Environment variables (as-built)

Variable	Default	Purpose
`ANTHROPIC_API_KEY`	empty	Claude API key. Empty means recorded mode.
`LLM_MODE`	`recorded`	`live` calls Claude by default, if there is a key.
`CLAUDE_MODEL`	`claude-opus-5-5`	Model for investigations and RCAs.
`LIVE_ACCESS_CODE`	empty	Lets a run opt into live while the default stays recorded.
`LIVE_DAILY_LIMIT`	`20`	Live model calls per UTC day.
`DEMO_MODE`	`true`	Enables demo reset.
`DEMO_RESET_ANY_IP`	`false`	Allows reset from any address. Docker only.
`DB_PATH`	`data/resolveai.db`	SQLite file, relative to the repository root.
`HOST`, `API_PORT`	`127.0.0.1`, `8787`	Listen address and port.
`MCP_ENABLED`	`true`	Start the MCP systems.
`MCP_MAX_FOLLOWUP_CALLS`	`8`	Follow-up calls per investigation, 0 to 20.
`RESOLVEAI_MCP_CONFIG`	empty	Config path. Otherwise `mcp.config.json`, then the example.
`MSSQL_USER`, `MSSQL_PASSWORD`	empty	SQL login, for the `tedious` driver setting only.

A.2 Glossary

Baseline check	A fixed read-only check ResolveAI runs for every investigation, before any model is involved.
Circuit	Blazor Server's live connection between one browser page and the server.
Evidence packet	The bounded, redacted set of records the model may see and cite.
FTS5	SQLite's full-text search engine. It ranks results with bm25.
Follow-up check	An extra tool call chosen by Claude (live), or replayed (recorded).
Golden file	A recorded correct output that tests compare against.
Hypothesis	A candidate root cause, with citations, counter-evidence, unknowns and checks.
MCP	Model Context Protocol: a standard way of offering tools to a model.
RCA	Root cause analysis: the post-incident report.
Recorded mode	Runs the pipeline with written or rule-based analysis and no model.
Speculative	A hypothesis with no valid citation; forced to low confidence.

A.3 Sample evidence packet item


```
{
  "id": "log:92d294db-9e68-5532-aadd-2ad515c20866",
  "type": "log",
  "timestamp": "2026-10-25T09:31:00Z",
  "source": "cdn/checkout-web",
  "excerpt": "ERROR cdn: GET /static/app.9f2c.js 404 [signature=cdn_asset_404]"
}
```


A.4 Sample tool result

A `db_blocking` result from the golden test cases, shortened:


```
{
  "summary": "1 blocked request(s) behind 1 head blocker(s) (session 61); longest wait 42s.",
  "rows": [
    { "role": "head_blocker", "session_id": 61, "status": "sleeping", "open_transaction_count": 1,
      "program_name": "SQLCMD",
      "last_query_text": "UPDATE dbo.Orders SET ReconciledAt = SYSUTCDATETIME() WHERE Status = N'open'" },
    { "role": "blocked", "session_id": 72, "blocking_session_id": 61, "database_name": "ResolveAIDemo",
      "wait_type": "LCK_M_U", "wait_ms": 42000, ... }
  ],
  ...
}
```